

Herald



Protocol Research: AMQP (1.0 + 0-9-1)

Field	Value
Revision	1
Created	2026-05-22
Last modified	2026-05-22
Status	research-only (no implementation yet)
Status summary	AMQP — Advanced Message Queuing Protocol — exists in two materially-different versions: AMQP 0-9-1 (RabbitMQ's original protocol; binary-framed; widely used in enterprises) and AMQP 1.0 (OASIS standard; supported natively in RabbitMQ 4.0+, Azure Service Bus, Apache Qpid). For Herald, AMQP is a P3 INGRESS protocol — Herald connects to an existing broker (RabbitMQ, Solace, Azure Service Bus, Qpid) and ingests messages. NOT recommended for Herald to RUN a broker. P3 opt-in.
Issues	open-questions: AMQP 0-9-1 (broader Go ecosystem) or AMQP 1.0 (standardized)?; exchange vs queue topology.
Continuation	Wave 4f+ — open HRD-327..HRD-334 only if operator wants AMQP ingest.

Constitutional anchors

- **§107 anti-bluff** — tests boot real RabbitMQ via `containers/`; `real rabbitmqctl` publishes; Herald consumes; assert receive.
- **§11.4.74 catalogue-check** — no `vasic-digital` or `HelixDevelopment` AMQP module. Use `github.com/rabbitmq/amqp091-go` (RabbitMQ team-maintained, AMQP 0-9-1) for primary; AMQP 1.0 via `github.com/Azure/go-amqp` if 1.0 chosen.
- **§11.4.61** — tracked doc.

Table of contents

- [§1. Protocol overview](#)
- [§2. Specification deep-dive](#)
- [§3. Herald-specific applicability analysis](#)
- [§4. Step-by-step implementation guide for Herald](#)
- [§5. §107 anti-bluff testing strategy](#)
- [§6. Open questions for operator](#)

- [§7. References](#)
- [§8. Catalogue-check verdict](#)

§1. Protocol overview

AMQP — Advanced Message Queuing Protocol. Born at JPMorgan Chase (2003); standardized by OASIS as AMQP 1.0 (2012). Two distinct wire protocols share the name:

- **AMQP 0-9-1** — predates the OASIS work. Used by RabbitMQ (its original protocol). Exchange + queue model (vs 1.0's link/session model). Most "AMQP Go library" usage in production is 0-9-1.
- **AMQP 1.0** — OASIS standard. Different wire format. Native in Azure Service Bus, Apache Qpid, Solace, ActiveMQ. Supported in RabbitMQ 4.0+ alongside 0-9-1.

Transport. TCP/IP (default port 5672; TLS 5671). Frame-based binary.

Authentication. SASL — PLAIN, EXTERNAL (mTLS), ANONYMOUS, EXTERNAL, OAuth.

Reliability. Both versions provide reliable delivery via per-message acknowledgments, publisher confirms, transactional consumption.

Topology (0-9-1). Exchanges (direct/topic/fanout/headers) route to queues; consumers subscribe to queues.

Topology (1.0). Symmetric peers exchange via "links" on "sessions" on "connections."

§2. Specification deep-dive

§2.1 AMQP 0-9-1 — exchange types

- **direct** — exact routing key match → queue.
- **topic** — wildcard routing key (* one word, # zero+ words).
- **fanout** — broadcast to all bound queues (ignores routing key).
- **headers** — route by AMQP headers (rare).

§2.2 AMQP 0-9-1 — acknowledgment

- `basic.ack` — single-message ack.
- `basic.nack` — neg-ack with optional requeue.
- `basic.reject` — reject single message.

Publisher confirms: `confirm.select` mode → server emits `basic.ack` for each delivered publish.

§2.3 AMQP 1.0 — links + flow control

- **Sender link** — publisher's transmit channel.
- **Receiver link** — subscriber's receive channel.
- **Credit-based flow control** — receiver advertises credit; sender uses it.

§2.4 Reliable delivery — patterns

- **At-most-once** — auto-ack + no publisher confirm.
- **At-least-once** — manual ack + no publisher confirm. Possible duplicates.
- **Effectively exactly-once** — manual ack + publisher confirm + receiver-side idempotency.

§2.5 Dead-letter exchange (DLX) — 0-9-1

Queue can be configured with DLX — messages exceeding redelivery threshold or rejected with no-requeue go to DLX → typically a "deadletter" queue for investigation.

§2.6 CloudEvents AMQP binding

CloudEvents v1.0.2 binding for AMQP 1.0: attributes → application properties prefixed `cloudEvents:`. Body = CloudEvent data.

§3. Herald-specific applicability analysis

§3.1 Use case

Herald INGESTS messages from AMQP brokers. Example:

- Banking system publishes transaction events to RabbitMQ exchange `transactions.events`.
- Herald binds a queue `herald.transactions` to that exchange.
- Each message becomes an event, dispatched.

Compared to Kafka: AMQP is "broker manages routing"; Kafka is "consumer manages offset." Different mental model.

§3.2 Herald is a CLIENT, not a broker

Herald should NOT run RabbitMQ. Use the existing org broker.

§3.3 0-9-1 or 1.0?

- 0-9-1 — broader Go ecosystem; `github.com/rabbitmq/amqp091-go` is mature + RabbitMQ-team maintained.
- 1.0 — more standardized; needed for Azure Service Bus / Solace; smaller Go ecosystem.

Recommendation: start with **0-9-1** (most users have RabbitMQ); add 1.0 in a Wave 4f+1 if operator demands Solace/Azure SB.

§3.4 Multi-tenant

Per-tenant connection + per-tenant queue. Each tenant configures broker URL + creds + exchange/queue bindings.

§3.5 Idempotency

AMQP can deliver duplicates on reconnect/redelivery. Use payload-embedded UUID or AMQP `message-id` as idempotency key.

§3.6 OTel propagation

CloudEvents traceparent extension → AMQP application property
`cloudEvents:traceparent`. Auto-extracted by Herald.

§3.7 Failure modes

1. **Broker disconnect.** Auto-reconnect with backoff. `amqp091-go` has built-in support but requires careful channel re-establishment.
2. **Channel exception.** AMQP channels close on any protocol violation. Recreate channel.
3. **Dead-letter overflow.** Monitor DLX; alert.
4. **Connection thrash.** Limit reconnect rate to avoid broker DoS.

§4. Step-by-step implementation guide for Herald

§4.1 Add dep

Submodule: `github.com/rabbitmq/amqp091-go` under `submodules/go-amqp091/`.

For AMQP 1.0 (if chosen): `github.com/Azure/go-amqp` under `submodules/go-amqp10/`.

§4.2 New channel adapter

`commons_messaging/channels/amqp/` — implements Channel interface; supports ingest (Subscribe) + egress (Send).

§4.3 Subscriber config

- `amqp_url` — `amqps://user:pass@broker.example.com:5671/vhost`.
- `amqp_exchange`, `amqp_routing_key`, `amqp_queue_name`.
- `amqp_qos_prefetch` — max in-flight.

§4.4 Egress

Herald publishes to AMQP topics as a fan-out channel.

§4.5 New `e2e_bluff_hunt` invariants

- **E98** — boot RabbitMQ via `containers/rabbitmqadmin publish ...`; Herald consumes; assert event in `events_processed`.
- **E99** — publisher confirms: Herald publishes with `confirm.select`; `basic.ack` received.
- **E100** — DLX: send unacked message; assert lands in DLX queue.
- **E101** — disconnect/reconnect: kill RabbitMQ; restart; assert resubscribe.
- **E102** — CloudEvents AMQP binding: publish with `cloudEvents:*` properties; assert `ce-id` propagates.

§4.6 HRD scaffolding

- **HRD-327** — vendor amqp091-go.
- **HRD-328** — channel adapter commons_messaging/channels/amqp/.
- **HRD-329** — reconnect + channel rebuild.
- **HRD-330** — DLX setup.
- **HRD-331** — egress publisher.
- **HRD-332** — CloudEvents AMQP binding.
- **HRD-333** — e2e_bluff_hunt E98–E102 + mutation gates.
- **HRD-334** — close Wave 4f (AMQP slice).

§5. §107 anti-bluff testing strategy

The bar: real RabbitMQ container; real CLI publishes; Herald consumes; event in DB.

§5.1 Happy-path

- **Test 1: consume.** Publish via rabbitmqadmin to exchange test; Herald subscribed; assert events_processed row.
- **Test 2: publish.** Herald notify channel amqp → external consumer receives.
- **Test 3: CloudEvents.** Publish with CE binding; ce-id preserved.

§5.2 Reliability

- **Test 4: ack/nack.** Process message; ack; not redelivered. nack; redelivered.
- **Test 5: publisher confirm.** Publish in confirm mode; assert ack received.
- **Test 6: DLX.** Nack with requeue=false → message in DLX queue.

§5.3 Failure modes

- **Test 7: broker down.** Kill; restart; resubscribe.
- **Test 8: bad creds.** SASL fails → backoff.
- **Test 9: channel exception.** Force invalid publish → channel closes; recreate.

§5.4 Performance

- **Test 10: throughput.** 10K msgs/sec sustained; lag < 1s.

§5.5 Mutation gates

- Mutation: skip reconnect → Test 7 FAILS.
- Mutation: skip publisher confirm → Test 5 FAILS.
- Mutation: skip DLX config → Test 6 FAILS.

§5.6 Wire-level

- tcpdump :5672 → AMQP frames; verify CONNECT + QoS + DELIVER sequence with Wireshark AMQP dissector.

§6. Open questions for operator

1. **In scope for MVP?** Recommend NO; opt-in.
2. **AMQP 0-9-1 only, 1.0 only, or both?** Recommend 0-9-1 first.
3. **Per-tenant connection or shared?** Per-tenant.
4. **Ingest only or also egress?** Both.
5. **Reuse RabbitMQ from containers / for tests?** Yes — add Mosquitto-style helper.

§7. References

(All fetched 2026-05-22.)

- **AMQP 1.0 spec.** <https://www.amqp.org/specification/1.0/amqp-org-download>
- **AMQP 0-9-1 RabbitMQ docs.** <https://www.rabbitmq.com/tutorials/amqp-concepts>
- **AMQP 1.0 in RabbitMQ.** <https://www.rabbitmq.com/docs/amqp>
- **amqp091-go.** <https://github.com/rabbitmq/amqp091-go>
- **go-amqp (AMQP 1.0).** <https://github.com/Azure/go-amqp>
- **CloudEvents AMQP binding.** <https://github.com/cloudevents/spec/blob/v1.0.2/cloudevents/bindings/amqp-protocol-binding.md>
- **Watermill AMQP pub/sub.** <https://watermill.io/pubsubs/amqp/>

§8. Catalogue-check verdict

- **vasic-digital:** no module.
- **HelixDevelopment:** no module.

Verdict: no-match → vendor `github.com/rabbitmq/amqp091-go` as **submodule**.
Optional second submodule for `github.com/Azure/go-amqp` if AMQP 1.0 is chosen.